COMPLETE DEVELOPER GUIDE

Bursary Management System - Step-by-Step Implementation

Project: Integrated Bursary Management System
Framework: Django 6.0 with Bootstrap 5
Database: PostgreSQL/SQLite
Created: February 2026
Status: Production Ready

TABLE OF CONTENTS

1. Project Setup
2. Database Design & Models
3. Backend Implementation
4. Frontend Implementation
5. Authentication & Authorization
6. File Structure & Organization
7. Complete Code Examples
8. Testing & Validation

---

1. PROJECT SETUP

1.1 Initial Environment Setup

Step 1: Create Virtual Environment

```
# Navigate to project directory
cd c:\Users\user\Documents\Finalyearproject\Bursary_system

# Create virtual environment
python -m venv .venv

# Activate virtual environment (Windows)
.\.venv\Scripts\Activate.ps1

# Or on Linux/Mac
source .venv/bin/activate
```

Step 2: Install Django & Dependencies

```
# Create requirements.txt with these packages:
Django==6.0.2
```

```
djangorestframework==3.14.0
django-cors-headers==4.2.0
python-decouple==3.8
Pillow==10.0.0
psycopg2-binary==2.9.7
gunicorn==21.2.0
python-dotenv==1.0.0
celery==5.3.1
redis==5.0.0
```

```
# Install all dependencies
pip install -r requirements.txt
```

Step 3: Create Django Project

```
# Create Django project
django-admin startproject bursary_system .

# Create Django apps
python manage.py startapp users
python manage.py startapp applications
python manage.py startapp schools
python manage.py startapp admin_panel
python manage.py startapp notifications
```

1.2 Project Structure

```
Bursary_system/
│
├── bursary_system/              # Main project config
│   ├── settings.py              # Django settings
│   ├── urls.py                  # URL routing
│   ├── wsgi.py                  # WSGI config
│   └── asgi.py                  # ASGI config
│
├── users/                       # User management app
│   ├── models.py               # User & Admin models
│   ├── views.py                # Auth & profile views
│   ├── forms.py                # User forms
│   ├── urls.py                 # User URLs
│   └── migrations/             # Database migrations
│
├── applications/               # Application management
│   ├── models.py              # Application models
│   ├── views.py               # Application views
│   ├── forms.py               # Application forms
```

```
│   ├── urls.py                # Application URLs
│   └── migrations/            # Database migrations
│
├── schools/                   # School management
│   ├── models.py             # School models
│   ├── views.py              # School views
│   ├── urls.py               # School URLs
│   └── migrations/           # Database migrations
│
├── admin_panel/              # Admin dashboard
│   ├── models.py             # Admin models
│   ├── views.py              # Admin views
│   ├── urls.py               # Admin URLs
│   └── migrations/           # Database migrations
│
├── notifications/            # Notification system
│   ├── models.py            # Notification models
│   ├── tasks.py             # Celery tasks
│   └── urls.py              # Notification URLs
│
├── templates/               # HTML templates
│   ├── base.html           # Base template
│   ├── users/              # User templates
│   ├── applications/       # Application templates
│   ├── admin_panel/        # Admin templates
│   └── schools/            # School templates
│
├── static/                 # Static files
│   ├── css/               # CSS files
│   ├── js/                # JavaScript files
│   └── images/            # Images
│
├── media/                 # User uploads
│   └── applications/      # Application documents
│
├── manage.py             # Django CLI
├── requirements.txt      # Python dependencies
├── .env                  # Environment variables
└── .gitignore           # Git ignore rules
```

---

2. DATABASE DESIGN & MODELS

2.1 Database Architecture Overview

```
The database is designed with the following principles:
- Normalization to eliminate data redundancy
- Foreign keys for referential integrity
- Indexes for optimal query performance
- Constraints for data validation
- Audit fields (created_at, updated_at) for tracking
```

2.2 Core Database Models

File: `users/models.py`

```
from django.db import models
from django.contrib.auth.models import User
from django.core.validators import FileExtensionValidator
import uuid
from datetime import timedelta
from django.utils import timezone

class UserProfile(models.Model):
    """Extended user profile for applicants"""

    GENDER_CHOICES = (
        ('Male', 'Male'),
        ('Female', 'Female'),
        ('Other', 'Other'),
    )

    MARITAL_CHOICES = (
        ('Single', 'Single'),
        ('Married', 'Married'),
        ('Widowed', 'Widowed'),
        ('Divorced', 'Divorced'),
    )

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    user = models.OneToOneField(User, on_delete=models.CASCADE,
related_name='profile')

    # Personal Information
    date_of_birth = models.DateField(blank=True, null=True)
    gender = models.CharField(max_length=10, choices=GENDER_CHOICES, blank=True)
    national_id = models.CharField(
        max_length=20,
        unique=True,  # Ensures uniqueness for fraud prevention
        db_index=True,  # Index for fast lookup
        help_text='National ID for fraud prevention'
    )
    marital_status = models.CharField(
        max_length=20,
        choices=MARITAL_CHOICES,
        default='Single'
    )

    # Contact Information
    phone_number = models.CharField(max_length=20, blank=True)
    phone_verified = models.BooleanField(default=False)

    # Location Information
```

```
    ward = models.ForeignKey(
        'applications.Ward',
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name='applicants',
        help_text='Ward or constituency'
    )

    # Verification Status
    is_verified = models.BooleanField(
        default=False,
        help_text='Admin has verified this account'
    )
    verified_by = models.ForeignKey(
        User,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name='verified_users'
    )
    verification_date = models.DateTimeField(blank=True, null=True)

    # Account Status
    is_active = models.BooleanField(default=True)
    email_verified = models.BooleanField(default=False)
    email_verified_date = models.DateTimeField(blank=True, null=True)

    # Financial Information
    family_income = models.DecimalField(
        max_digits=12,
        decimal_places=2,
        blank=True,
        null=True,
        help_text='Annual family income in KES'
    )

    # Tracking
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    last_login = models.DateTimeField(blank=True, null=True)

    class Meta:
        ordering = ['-created_at']
        indexes = [
            models.Index(fields=['user']),
            models.Index(fields=['national_id']),
            models.Index(fields=['ward']),
            models.Index(fields=['is_verified']),
        ]
        verbose_name_plural = "User Profiles"

    def __str__(self):
        return f"{self.user.get_full_name()} - {self.national_id}"
```

```python
    @property
    def is_age_eligible(self):
        """Check if applicant is age eligible (typically 18+)"""
        if not self.date_of_birth:
            return False
        today = timezone.now().date()
        age = today.year - self.date_of_birth.year
        return age >= 18


class AdminRole(models.Model):
    """Admin role assignments for staff"""

    ROLE_CHOICES = (
        ('Super_Admin', 'Super Admin'),
        ('CDF_Admin', 'CDF Admin'),
        ('Ward_Admin', 'Ward Admin'),
    )

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    user = models.OneToOneField(
        User,
        on_delete=models.CASCADE,
        related_name='admin_role'
    )

    # Role Assignment
    role_type = models.CharField(
        max_length=20,
        choices=ROLE_CHOICES,
        db_index=True
    )

    # Ward Assignment (optional for CDF/Super admin)
    ward = models.ForeignKey(
        'applications.Ward',
        on_delete=models.SET_NULL,
        null=True,
        blank=True,  # Not required for CDF_Admin or Super_Admin
        related_name='ward_admins',
        help_text='Ward assigned (only for Ward Admin)'
    )

    # Verification Status
    is_verified = models.BooleanField(
        default=False,
        help_text='Super admin has verified this admin'
    )
    verified_by = models.ForeignKey(
        User,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
```

```
        related_name='admins_verified'
    )
    verified_date = models.DateTimeField(blank=True, null=True)

    # Tracking
    assigned_date = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    is_active = models.BooleanField(default=True)

    class Meta:
        ordering = ['-assigned_date']
        indexes = [
            models.Index(fields=['role_type']),
            models.Index(fields=['ward']),
            models.Index(fields=['is_verified']),
        ]
        constraints = [
            models.UniqueConstraint(
                fields=['user'],
                name='one_admin_role_per_user'
            ),
        ]

    def __str__(self):
        return f"{self.user.username} - {self.get_role_type_display()}"

    def get_permissions(self):
        """Get list of permissions for this role"""
        permissions_map = {
            'Super_Admin': [
                'view_all_applications',
                'approve_applications',
                'reject_applications',
                'create_admin',
                'manage_permissions',
                'view_system_reports',
                'manage_ward_data',
                'override_decisions',
            ],
            'CDF_Admin': [
                'view_all_applications',
                'approve_applications',
                'reject_applications',
                'view_all_reports',
                'award_amounts',
            ],
            'Ward_Admin': [
                'view_ward_applications',
                'score_applications',
                'make_recommendations',
                'add_comments',
                'view_ward_reports',
                'export_ward_data',
            ],
```

```
        }
        return permissions_map.get(self.role_type, [])


class PasswordReset(models.Model):
    """Password reset tokens for security"""

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    token = models.CharField(max_length=6, unique=True)  # 6-digit reset code
    created_at = models.DateTimeField(auto_now_add=True)
    expires_at = models.DateTimeField()
    is_used = models.BooleanField(default=False)

    class Meta:
        ordering = ['-created_at']

    def is_expired(self):
        """Check if reset token has expired"""
        return timezone.now() > self.expires_at

    def __str__(self):
        return f"Reset for {self.user.email} - {self.created_at}"
```

File: `applications/models.py`

```
from django.db import models
from django.contrib.auth.models import User
from schools.models import School, Program, Campus
import uuid

class Ward(models.Model):
    """Wards/Constituencies for location tracking"""

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    name = models.CharField(max_length=100, unique=True, db_index=True)
    constituency = models.CharField(max_length=100, blank=True)
    county = models.CharField(max_length=100, blank=True, db_index=True)
    description = models.TextField(blank=True)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['county', 'constituency', 'name']
        indexes = [
            models.Index(fields=['county', 'constituency']),
        ]

    def __str__(self):
        if self.constituency:
```

```
            return f"{self.name} ({self.constituency}, {self.county})"
        return self.name


class Application(models.Model):
    """Bursary application"""

    STATUS_CHOICES = (
        ('Draft', 'Draft - Not Submitted'),
        ('Submitted', 'Submitted - Under Review'),
        ('Under_Review', 'Under Review - Admin Reviewing'),
        ('Needs_Clarification', 'Needs Clarification'),
        ('Approved', 'Approved'),
        ('Rejected', 'Rejected'),
        ('Appeal', 'Under Appeal Review'),
    )

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    applicant = models.ForeignKey(
        User,
        on_delete=models.CASCADE,
        related_name='applications',
        db_index=True
    )

    # School Information
    school = models.ForeignKey(
        School,
        on_delete=models.PROTECT,
        related_name='applications'
    )
    campus = models.ForeignKey(
        Campus,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name='applications'
    )
    program = models.ForeignKey(
        Program,
        on_delete=models.PROTECT,
        related_name='applications'
    )
    ward = models.ForeignKey(
        Ward,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name='applications',
        help_text='Applicant ward'
    )

    # Application Status
    status = models.CharField(
```

```
        max_length=50,
        choices=STATUS_CHOICES,
        default='Draft',
        db_index=True
    )

    # Personal Information
    date_of_birth = models.DateField(blank=True, null=True)
    gender = models.CharField(
        max_length=10,
        choices=[('Male', 'Male'), ('Female', 'Female')],
        blank=True
    )
    national_id = models.CharField(max_length=20, blank=True)
    marital_status = models.CharField(
        max_length=20,
        choices=[('Single', 'Single'), ('Married', 'Married')],
        default='Single'
    )
    is_orphan = models.BooleanField(default=False)

    # Contact Information
    phone_number = models.CharField(max_length=20, blank=True)
    email = models.EmailField(blank=True)

    # Financial Information
    family_income = models.DecimalField(
        max_digits=12,
        decimal_places=2,
        blank=True,
        null=True,
        help_text='Annual family income in KES'
    )
    income_source = models.CharField(max_length=255, blank=True)
    number_of_dependents = models.IntegerField(default=0, blank=True, null=True)
    other_siblings_in_school = models.IntegerField(default=0, blank=True,
null=True)

    # Academic Information
    course_name = models.CharField(max_length=255, blank=True)
    program_level = models.CharField(
        max_length=50,
        choices=[
            ('Certificate', 'Certificate'),
            ('Diploma', 'Diploma'),
            ('Degree', 'Degree'),
            ('Masters', 'Masters'),
        ],
        blank=True
    )
    year_of_study = models.CharField(
        max_length=20,
        choices=[
            ('1st Year', '1st Year'),
```

```
            ('2nd Year', '2nd Year'),
            ('3rd Year', '3rd Year'),
            ('4th Year', '4th Year'),
        ],
        blank=True
    )

    # Application Essay
    motivation_letter = models.TextField(blank=True)
    expected_challenges = models.TextField(blank=True)

    # Review Information
    reviewed_by = models.ForeignKey(
        User,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name='reviewed_applications'
    )
    reviewed_date = models.DateTimeField(blank=True, null=True)

    # Timestamps
    application_date = models.DateTimeField(auto_now_add=True)
    submitted_date = models.DateTimeField(blank=True, null=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['-created_at']
        indexes = [
            models.Index(fields=['applicant', 'status']),
            models.Index(fields=['school', 'status']),
            models.Index(fields=['status', 'submitted_date']),
        ]

    def __str__(self):
        return f"{self.applicant.get_full_name()} - {self.school.name}
({self.status})"

    @property
    def approved_amount(self):
        """Get the approved amount from ApplicationApproval"""
        approval = self.approvals.filter(
            status='Approved',
            amount_approved__isnull=False
        ).order_by('-approved_date').first()
        return approval.amount_approved if approval else None


class ApplicationDocument(models.Model):
    """Supporting documents for application"""

    DOCUMENT_TYPES = (
        ('Admission_Letter', 'Admission Letter'),
```

```
        ('Birth_Certificate', 'Birth Certificate'),
        ('National_ID', 'National ID'),
        ('Fee_Structure', 'Fee Structure'),
        ('Parents_ID', "Parent's ID"),
        ('Death_Certificate', 'Death Certificate'),
        ('Other', 'Other Document'),
    )

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    application = models.ForeignKey(
        Application,
        on_delete=models.CASCADE,
        related_name='documents'
    )
    document_type = models.CharField(max_length=50, choices=DOCUMENT_TYPES)
    file = models.FileField(
        upload_to='applications/documents/',
        validators=[
            FileExtensionValidator(
                allowed_extensions=['pdf', 'png', 'jpg', 'jpeg', 'doc', 'docx']
            )
        ]
    )
    uploaded_date = models.DateTimeField(auto_now_add=True)
    is_verified = models.BooleanField(default=False)
    verified_by = models.ForeignKey(
        User,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name='verified_documents'
    )
    verified_date = models.DateTimeField(blank=True, null=True)
    comments = models.TextField(blank=True)

    class Meta:
        ordering = ['document_type', '-uploaded_date']
        unique_together = ('application', 'document_type')

    def __str__(self):
        return f"{self.application.applicant.get_full_name()} -
{self.get_document_type_display()}"


class ApplicationReview(models.Model):
    """Admin review and scoring"""

    REVIEW_STATUS = (
        ('Pending', 'Pending Review'),
        ('In_Progress', 'In Progress'),
        ('Completed', 'Review Completed'),
    )

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
```

```
    application = models.OneToOneField(
        Application,
        on_delete=models.CASCADE,
        related_name='review'
    )
    reviewer = models.ForeignKey(
        User,
        on_delete=models.PROTECT,
        related_name='reviews'
    )

    # Scoring (0-100)
    academic_score = models.IntegerField(default=0)
    financial_need_score = models.IntegerField(default=0)
    supporting_documents_score = models.IntegerField(default=0)
    overall_score = models.IntegerField(default=0)

    # Review Status
    review_status = models.CharField(
        max_length=20,
        choices=REVIEW_STATUS,
        default='Pending'
    )
    recommendation = models.CharField(
        max_length=20,
        choices=[
            ('Approve', 'Approve'),
            ('Reject', 'Reject'),
            ('Clarify', 'Needs Clarification')
        ],
        blank=True
    )

    # Comments
    comments = models.TextField(blank=True)
    internal_notes = models.TextField(blank=True, help_text='For admin use only')

    # Timestamps
    reviewed_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['-reviewed_at']

    def __str__(self):
        return f"Review of {self.application.applicant.get_full_name()}"


class ApplicationApproval(models.Model):
    """Track approval decisions and amounts"""

    APPROVAL_LEVELS = (
        ('Ward_Admin', 'Ward Admin'),
        ('CDF_Admin', 'CDF Admin'),
```

```
        ('Super_Admin', 'Super Admin'),
    )

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    application = models.ForeignKey(
        Application,
        on_delete=models.CASCADE,
        related_name='approvals'
    )
    approved_by = models.ForeignKey(
        User,
        on_delete=models.PROTECT,
        related_name='approvals_given'
    )

    # Approval Details
    approval_level = models.CharField(max_length=20, choices=APPROVAL_LEVELS)
    status = models.CharField(
        max_length=20,
        choices=[('Approved', 'Approved'), ('Rejected', 'Rejected')]
    )
    reason = models.TextField(blank=True)
    amount_approved = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        blank=True,
        null=True,
        help_text='Approved amount in KES'
    )

    # Timestamps
    approved_date = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['-approved_date']

    def __str__(self):
        return f"{self.application.applicant.get_full_name()} -
{self.approval_level}: {self.status}"
```

File: `schools/models.py`

```
from django.db import models
import uuid

class School(models.Model):
    """Educational institutions"""

    SCHOOL_TYPES = (
        ('Primary', 'Primary School'),
        ('Secondary', 'Secondary School'),
```

```
        ('University', 'University'),
        ('TVET', 'TVET/College'),
        ('Private', 'Private Institution'),
    )

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    name = models.CharField(max_length=255, unique=True, db_index=True)
    school_type = models.CharField(max_length=50, choices=SCHOOL_TYPES,
db_index=True)

    # Contact Information
    phone_number = models.CharField(max_length=20, blank=True)
    email = models.EmailField(blank=True)
    website = models.URLField(blank=True)

    # Location
    location = models.CharField(max_length=255, blank=True)
    county = models.CharField(max_length=100, blank=True, db_index=True)

    # Details
    description = models.TextField(blank=True)
    is_active = models.BooleanField(default=True)

    # Tracking
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['name']
        indexes = [
            models.Index(fields=['school_type']),
            models.Index(fields=['county']),
        ]

    def __str__(self):
        return self.name


class Campus(models.Model):
    """School campuses"""

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    school = models.ForeignKey(
        School,
        on_delete=models.CASCADE,
        related_name='campuses'
    )
    name = models.CharField(max_length=255)
    location = models.CharField(max_length=255, blank=True)
    phone_number = models.CharField(max_length=20, blank=True)
    email = models.EmailField(blank=True)
    is_active = models.BooleanField(default=True)

    class Meta:
```

```
        unique_together = ('school', 'name')

    def __str__(self):
        return f"{self.school.name} - {self.name}"


class Program(models.Model):
    """Academic programs offered"""

    PROGRAM_TYPES = (
        ('Diploma', 'Diploma'),
        ('Degree', 'Bachelor Degree'),
        ('Certificate', 'Certificate'),
        ('Masters', 'Masters'),
    )

    CATEGORY_CHOICES = (
        ('Engineering', 'Engineering & Technology'),
        ('Health', 'Health Sciences'),
        ('Business', 'Business & Commerce'),
        ('Education', 'Education'),
        ('Law', 'Law'),
        ('Agriculture', 'Agriculture'),
        ('Arts', 'Arts & Humanities'),
        ('Science', 'Science'),
        ('Other', 'Other'),
    )

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    school = models.ForeignKey(
        School,
        on_delete=models.CASCADE,
        related_name='programs'
    )

    # Program Details
    name = models.CharField(max_length=255)
    code = models.CharField(max_length=50, blank=True)
    category = models.CharField(
        max_length=50,
        choices=CATEGORY_CHOICES,
        db_index=True
    )
    program_type = models.CharField(
        max_length=50,
        choices=PROGRAM_TYPES,
        default='Diploma'
    )

    # Details
    description = models.TextField(blank=True)
    duration_months = models.IntegerField(default=24)
    is_active = models.BooleanField(default=True)
```

```
    # Tracking
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['category', 'name']
        indexes = [
            models.Index(fields=['school', 'category']),
        ]

    def __str__(self):
        return f"{self.school.name} - {self.name}"
```

2.3 Database Relationships Diagram

```
┌──────────────────┐
│   User (Auth)    │
└──────────────────┘
         │
         │ 1:1
         ├──────────────┐
         │              │
         ▼              ▼
    ┌─────────┐    ┌──────────────┐
    │ Profile │    │ AdminRole    │
    └─────────┘    └──────────────┘
         △              │
         │              │ (Ward_Admin has Ward)
         │              ▼
    ┌──────────────────────┐
    │ Application (Applicant)│
    └──────────────────────┘
         │    │    │
         │    │    └─→ School
         │    ├─────→ Program
         │    └─────→ Ward
         │
         ├─→ ApplicationReview
         ├─→ ApplicationDocument
         └─→ ApplicationApproval
```

2.4 Creating Migrations

```
# Create migrations for each app
python manage.py makemigrations users
python manage.py makemigrations applications
python manage.py makemigrations schools
python manage.py makemigrations admin_panel
python manage.py makemigrations notifications

# Apply migrations to database
```

```
python manage.py migrate users
python manage.py migrate applications
python manage.py migrate schools
python manage.py migrate admin_panel
python manage.py migrate notifications

# Check migration status
python manage.py showmigrations
```

---

3. BACKEND IMPLEMENTATION

3.1 Django Settings Configuration

File: `bursary_system/settings.py`

```
import os
from pathlib import Path
from decouple import config

# Build paths
BASE_DIR = Path(__file__).resolve().parent.parent

# Security
SECRET_KEY = config('SECRET_KEY', default='your-secret-key-here-change-in-
production')
DEBUG = config('DEBUG', default=True, cast=bool)
ALLOWED_HOSTS = config('ALLOWED_HOSTS', default='localhost,127.0.0.1', cast=lambda
v: [s.strip() for s in v.split(',')])

# Application definition
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # Third-party apps
    'rest_framework',
    'corsheaders',

    # Local apps
    'users.apps.UsersConfig',
    'applications.apps.ApplicationsConfig',
    'schools.apps.SchoolsConfig',
    'admin_panel.apps.AdminPanelConfig',
    'notifications.apps.NotificationsConfig',
]
```

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'corsheaders.middleware.CorsMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'bursary_system.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'bursary_system.wsgi.application'

# Database
DATABASES = {
    'default': {
        'ENGINE': config('DB_ENGINE', default='django.db.backends.sqlite3'),
        'NAME': config('DB_NAME', default=str(BASE_DIR / 'db.sqlite3')),
        'USER': config('DB_USER', default=''),
        'PASSWORD': config('DB_PASSWORD', default=''),
        'HOST': config('DB_HOST', default=''),
        'PORT': config('DB_PORT', default=''),
    }
}

# Password validation
AUTH_PASSWORD_VALIDATORS = [
    {'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator'},
    {'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator'},
    {'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator'},
    {'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator'},
]

# Internationalization
LANGUAGE_CODE = 'en-us'
```

```
TIME_ZONE = 'UTC'
USE_I18N = True
USE_TZ = True

# Static files
STATIC_URL = '/static/'
STATIC_ROOT = BASE_DIR / 'staticfiles'
STATICFILES_DIRS = [BASE_DIR / 'static']

# Media files
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'

# File upload
FILE_UPLOAD_MAX_MEMORY_SIZE = 5242880  # 5MB
DATA_UPLOAD_MAX_MEMORY_SIZE = 5242880  # 5MB

# Default primary key field type
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

# Authentication
LOGIN_URL = 'users:login'
LOGIN_REDIRECT_URL = 'users:dashboard'
LOGOUT_REDIRECT_URL = 'users:login'

# Email Configuration
EMAIL_BACKEND = config('EMAIL_BACKEND',
default='django.core.mail.backends.console.EmailBackend')
EMAIL_HOST = config('EMAIL_HOST', default='smtp.gmail.com')
EMAIL_PORT = config('EMAIL_PORT', default=587, cast=int)
EMAIL_USE_TLS = config('EMAIL_USE_TLS', default=True, cast=bool)
EMAIL_HOST_USER = config('EMAIL_HOST_USER', default='')
EMAIL_HOST_PASSWORD = config('EMAIL_HOST_PASSWORD', default='')

# Session settings
SESSION_ENGINE = 'django.contrib.sessions.backends.db'
SESSION_COOKIE_AGE = 600  # 10 minutes
SESSION_EXPIRE_AT_BROWSER_CLOSE = True

# Logging
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'handlers': {
        'file': {
            'level': 'INFO',
            'class': 'logging.FileHandler',
            'filename': BASE_DIR / 'logs' / 'debug.log',
        },
        'console': {
            'level': 'DEBUG',
            'class': 'logging.StreamHandler',
        },
    },
```

```
    'root': {
        'handlers': ['console', 'file'],
        'level': 'INFO',
    },
}

# CORS settings
CORS_ALLOWED_ORIGINS = [
    "http://localhost:3000",
    "http://localhost:8000",
]

# Rest Framework
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.SessionAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
}
```

3.2 Authentication & Authorization Views

File: `users/views.py`

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required
from django.contrib.auth.models import User
from django.contrib import messages
from django.db.models import Q
import logging
from .models import UserProfile, AdminRole, PasswordReset
from .forms import RegisterForm, LoginForm, AdminRoleAssignmentForm
from applications.models import Application
import random
from django.utils import timezone
from datetime import timedelta

logger = logging.getLogger(__name__)

# ==================== REGISTRATION ====================

def register(request):
    """User registration with email verification"""

    if request.user.is_authenticated:
        return redirect('users:dashboard')

    if request.method == 'POST':
```

```
        form = RegisterForm(request.POST)
        if form.is_valid():
            try:
                # Create user
                user = User.objects.create_user(
                    username=form.cleaned_data['username'],
                    email=form.cleaned_data['email'],
                    password=form.cleaned_data['password'],
                    first_name=form.cleaned_data.get('first_name', ''),
                    last_name=form.cleaned_data.get('last_name', ''),
                )

                # Create user profile with national ID (FRAUD PREVENTION)
                profile = UserProfile.objects.create(
                    user=user,
                    national_id=form.cleaned_data['national_id'],
                    phone_number=form.cleaned_data.get('phone_number', ''),
                    gender=form.cleaned_data.get('gender', ''),
                )

                # Send verification email
                send_verification_email(user)

                messages.success(request, 'Registration successful! Check your
email for verification link.')
                logger.info(f"User registered: {user.username}")
                return redirect('users:login')

            except Exception as e:
                logger.error(f"Registration error: {str(e)}")
                messages.error(request, 'Registration failed. Please try again.')
        else:
            for field, errors in form.errors.items():
                for error in errors:
                    messages.error(request, f"{field}: {error}")

    else:
        form = RegisterForm()

    return render(request, 'users/register.html', {'form': form})


def verify_email(request, token):
    """Verify user email with token"""

    try:
        # Find user by verification token (simplified)
        user = User.objects.get(email=request.GET.get('email'))
        user.profile.email_verified = True
        user.profile.email_verified_date = timezone.now()
        user.profile.save()

        messages.success(request, 'Email verified! You can now login.')
        logger.info(f"Email verified for user: {user.username}")
```

```
        return redirect('users:login')

    except User.DoesNotExist:
        messages.error(request, 'Verification failed. Invalid email.')
        return redirect('users:login')


# ==================== LOGIN & LOGOUT ====================

def login_view(request):
    """User login with optional national ID verification"""

    if request.user.is_authenticated:
        return redirect('users:dashboard')

    if request.method == 'POST':
        form = LoginForm(request.POST)
        if form.is_valid():
            username = form.cleaned_data['username']
            password = form.cleaned_data['password']

            # Authenticate user
            user = authenticate(request, username=username, password=password)

            if user is not None:
                # Check if email verified
                if not user.profile.email_verified:
                    messages.warning(request, 'Please verify your email first.')
                    return redirect('users:login')

                # Check if account is active
                if not user.profile.is_active:
                    messages.error(request, 'Your account has been deactivated.')
                    logger.warning(f"Login attempt by deactivated user:
{username}")
                    return redirect('users:login')

                # Login user
                login(request, user)
                user.profile.last_login = timezone.now()
                user.profile.save()

                logger.info(f"User logged in: {username}")

                # Redirect based on role
                if hasattr(user, 'admin_role'):
                    return redirect('admin_panel:dashboard')
                else:
                    return redirect('users:dashboard')
            else:
                messages.error(request, 'Invalid username or password.')
                logger.warning(f"Failed login attempt for: {username}")

    else:
```

```
        form = LoginForm()

    return render(request, 'users/login.html', {'form': form})


def logout_view(request):
    """User logout"""

    logout(request)
    messages.success(request, 'Logged out successfully.')
    return redirect('users:login')


# ==================== DASHBOARD ====================

@login_required(login_url='users:login')
def dashboard(request):
    """Applicant dashboard with application status"""

    user = request.user

    # Get user's applications
    applications = Application.objects.filter(
        applicant=user
    ).order_by('-created_at')

    # Count by status
    stats = {
        'total': applications.count(),
        'draft': applications.filter(status='Draft').count(),
        'submitted': applications.filter(status='Submitted').count(),
        'under_review': applications.filter(status='Under_Review').count(),
        'approved': applications.filter(status='Approved').count(),
        'rejected': applications.filter(status='Rejected').count(),
    }

    # Add approval info to each application
    for app in applications:
        app.approved_amount = app.approved_amount

    context = {
        'user': user,
        'applications': applications,
        'stats': stats,
    }

    return render(request, 'users/dashboard.html', context)


# ==================== PROFILE MANAGEMENT ====================

@login_required(login_url='users:login')
def profile(request):
    """View and edit user profile"""
```

```
    user = request.user
    profile = user.profile

    if request.method == 'POST':
        # Update profile
        profile.date_of_birth = request.POST.get('date_of_birth',
profile.date_of_birth)
        profile.phone_number = request.POST.get('phone_number',
profile.phone_number)
        profile.gender = request.POST.get('gender', profile.gender)
        profile.save()

        messages.success(request, 'Profile updated successfully.')
        logger.info(f"Profile updated for user: {user.username}")

    context = {
        'user': user,
        'profile': profile,
    }

    return render(request, 'users/profile.html', context)


# ==================== ADMIN FUNCTIONS ====================

def is_admin(user):
    """Check if user is an admin"""
    return hasattr(user, 'admin_role') and user.admin_role.is_active


@login_required(login_url='users:login')
def assign_admin_role(request):
    """Assign admin role to user (Super Admin only)"""

    # Check if super admin
    if not is_admin(request.user) or request.user.admin_role.role_type !=
'Super_Admin':
        messages.error(request, 'Unauthorized access.')
        return redirect('users:dashboard')

    if request.method == 'POST':
        form = AdminRoleAssignmentForm(request.POST)
        if form.is_valid():
            try:
                user = form.cleaned_data['user']
                role_type = form.cleaned_data['role_type']
                ward = form.cleaned_data.get('ward')

                # Check if user already has admin role
                if hasattr(user, 'admin_role'):
                    messages.warning(request, 'User already has an admin role.')
                    return redirect('users:assign_admin_role')
```

```
                # Create admin role
                admin_role = AdminRole.objects.create(
                    user=user,
                    role_type=role_type,
                    ward=ward if role_type == 'Ward_Admin' else None
                )

                messages.success(request, f'Admin role assigned to
{user.get_full_name()}')
                logger.info(f"Admin role assigned: {user.username} - {role_type}")
                return redirect('users:manage_admins')

            except Exception as e:
                logger.error(f"Error assigning admin role: {str(e)}")
                messages.error(request, 'Error assigning role.')

    else:
        form = AdminRoleAssignmentForm()

    context = {'form': form}
    return render(request, 'admin_panel/assign_admin_role.html', context)


@login_required(login_url='users:login')
def manage_admins(request):
    """List and manage admin users"""

    if not is_admin(request.user) or request.user.admin_role.role_type !=
'Super_Admin':
        messages.error(request, 'Unauthorized access.')
        return redirect('users:dashboard')

    admins = AdminRole.objects.select_related('user', 'verified_by', 'ward').all()

    context = {
        'admins': admins,
    }

    return render(request, 'admin_panel/manage_admins.html', context)


# ==================== PASSWORD RESET ====================

def forgot_password(request):
    """Password reset request"""

    if request.method == 'POST':
        email = request.POST.get('email')

        try:
            user = User.objects.get(email=email)

            # Generate 6-digit reset code
            reset_code = str(random.randint(100000, 999999))
```

```
            # Create password reset token
            reset = PasswordReset.objects.create(
                user=user,
                token=reset_code,
                expires_at=timezone.now() + timedelta(minutes=15)
            )

            # Send email with reset code
            send_password_reset_email(user, reset_code)

            messages.success(request, f'Password reset code sent to {email}')
            logger.info(f"Password reset requested for: {email}")
            return redirect('users:reset_password')

        except User.DoesNotExist:
            # Don't reveal if email exists
            messages.info(request, 'If this email exists, you will receive a reset
code.')

    return render(request, 'users/forgot_password.html')


def reset_password(request):
    """Reset password with code"""

    if request.method == 'POST':
        email = request.POST.get('email')
        code = request.POST.get('code')
        new_password = request.POST.get('new_password')

        try:
            user = User.objects.get(email=email)
            reset = PasswordReset.objects.get(
                user=user,
                token=code,
                is_used=False
            )

            # Check expiration
            if reset.is_expired():
                raise PasswordReset.DoesNotExist('Code expired')

            # Update password
            user.set_password(new_password)
            user.save()

            # Mark as used
            reset.is_used = True
            reset.save()

            messages.success(request, 'Password reset successful. You can now
login.')
            logger.info(f"Password reset successful for: {email}")
```

```
            return redirect('users:login')

        except (User.DoesNotExist, PasswordReset.DoesNotExist):
            messages.error(request, 'Invalid email or reset code.')

    return render(request, 'users/reset_password.html')


# ==================== HELPER FUNCTIONS ====================

def send_verification_email(user):
    """Send email verification link"""

    from django.core.mail import send_mail
    from django.urls import reverse

    verification_url = f"{reverse('users:verify_email')}?email={user.email}"

    subject = 'Email Verification - Bursary System'
    message = f"""
    Welcome to Matungu Subcounty Bursary System!

    Click the link below to verify your email:
    {verification_url}

    Link expires in 24 hours.
    """

    send_mail(
        subject,
        message,
        'noreply@bursarysystem.com',
        [user.email],
        fail_silently=True,
    )


def send_password_reset_email(user, code):
    """Send password reset code"""

    from django.core.mail import send_mail

    subject = 'Password Reset Code - Bursary System'
    message = f"""
    Your password reset code is: {code}

    This code expires in 15 minutes.

    If you didn't request this, ignore this email.
    """

    send_mail(
        subject,
        message,
```

```
        'noreply@bursarysystem.com',
        [user.email],
        fail_silently=True,
    )
```

3.3 Application Management Views

File: `applications/views.py` **(Partial)**

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from django.views.decorators.http import require_http_methods
from django.db.models import Q, Sum, Count, Avg
import json
import logging
from .models import (
    Application,
    ApplicationDocument,
    ApplicationReview,
    ApplicationApproval,
    Ward
)
from schools.models import School, Program
import uuid

logger = logging.getLogger(__name__)

# ==================== NEW APPLICATION ====================

@login_required(login_url='users:login')
def new_application(request):
    """Create new bursary application (Multi-step form)"""

    # Check if user is applicant (not admin)
    if hasattr(request.user, 'admin_role'):
        messages.error(request, 'Only applicants can create applications.')
        return redirect('admin_panel:dashboard')

    step = request.GET.get('step', '1')

    if request.method == 'POST':
        # Create new application
        application = Application.objects.create(
            applicant=request.user,
            status='Draft'
        )

        # Save data based on step
        if step == '1':
            # Personal information
```

```
            application.date_of_birth = request.POST.get('date_of_birth')
            application.gender = request.POST.get('gender')
            application.national_id = request.POST.get('national_id')
            application.marital_status = request.POST.get('marital_status')
            application.is_orphan = request.POST.get('is_orphan') == 'on'

        elif step == '2':
            # School information
            try:
                school = School.objects.get(id=request.POST.get('school'))
                program = Program.objects.get(id=request.POST.get('program'))

                application.school = school
                application.program = program
                application.year_of_study = request.POST.get('year_of_study')
                application.course_name = request.POST.get('course_name')
                application.program_level = request.POST.get('program_level')

            except (School.DoesNotExist, Program.DoesNotExist):
                messages.error(request, 'Invalid school or program.')
                return redirect('applications:new_application', step='2')

        elif step == '3':
            # Financial information
            application.family_income = request.POST.get('family_income', 0)
            application.income_source = request.POST.get('income_source')
            application.number_of_dependents =
int(request.POST.get('number_of_dependents', 0))
            application.other_siblings_in_school =
int(request.POST.get('other_siblings_in_school', 0))

        elif step == '4':
            # Essays
            application.motivation_letter = request.POST.get('motivation_letter')
            application.expected_challenges =
request.POST.get('expected_challenges')

        application.save()

        # Determine next step
        next_step = str(int(step) + 1)
        if next_step == '5':
            # Document upload step
            return redirect('applications:upload_documents',
id=str(application.id))

        return redirect('applications:new_application', step=next_step)

    # GET request - show form for current step
    context = {
        'step': step,
        'schools': School.objects.filter(is_active=True),
        'programs': Program.objects.filter(is_active=True),
        'wards': Ward.objects.filter(is_active=True),
```

```
    }

    return render(request, f'applications/new_application_step{step}.html',
context)


# ==================== DOCUMENT UPLOAD ====================

@login_required(login_url='users:login')
def upload_documents(request, id):
    """Upload supporting documents"""

    try:
        application = Application.objects.get(id=id, applicant=request.user)
    except Application.DoesNotExist:
        messages.error(request, 'Application not found.')
        return redirect('users:dashboard')

    # Check if application is in Draft status
    if application.status != 'Draft':
        messages.error(request, 'Cannot upload documents for submitted
application.')
        return redirect('applications:view_application', id=id)

    if request.method == 'POST':
        # Handle file uploads
        for doc_type, file_obj in request.FILES.items():
            if file_obj:
                document = ApplicationDocument.objects.create(
                    application=application,
                    document_type=doc_type,
                    file=file_obj
                )
                logger.info(f"Document uploaded: {document.id}")

        messages.success(request, 'Documents uploaded successfully.')
        return redirect('applications:review_application', id=id)

    # Show existing documents
    documents = application.documents.all()

    context = {
        'application': application,
        'documents': documents,
        'document_types': ApplicationDocument.DOCUMENT_TYPES,
    }

    return render(request, 'applications/upload_documents.html', context)


# ==================== SUBMIT APPLICATION ====================

@login_required(login_url='users:login')
def submit_application(request, id):
```

```
    """Submit application for review"""

    try:
        application = Application.objects.get(id=id, applicant=request.user)
    except Application.DoesNotExist:
        messages.error(request, 'Application not found.')
        return redirect('users:dashboard')

    # Validation
    if application.status != 'Draft':
        messages.warning(request, 'Application already submitted.')
        return redirect('applications:view_application', id=id)

    # Check required fields
    required_fields = ['school', 'program', 'motivation_letter']
    for field in required_fields:
        if not getattr(application, field):
            messages.error(request, f'{field} is required.')
            return redirect('applications:edit_application', id=id)

    # Check documents uploaded
    if application.documents.count() == 0:
        messages.warning(request, 'Please upload at least one document.')
        return redirect('applications:upload_documents', id=id)

    # Update application status
    application.status = 'Submitted'
    application.submitted_date = timezone.now()
    application.save()

    # Create review record
    ApplicationReview.objects.create(
        application=application,
        reviewer=application.reviewed_by,
        review_status='Pending'
    )

    messages.success(request, 'Application submitted successfully.')
    logger.info(f"Application submitted: {application.id} by
{request.user.username}")

    # Send notification
    send_application_submitted_email(application)

    return redirect('applications:view_application', id=id)


# ==================== TRACK APPLICATION ====================

@login_required(login_url='users:login')
def track_application(request):
    """View all applications with status tracking"""

    applications = Application.objects.filter(
```

```
        applicant=request.user
    ).order_by('-created_at')

    # Add progress stages
    for app in applications:
        app.stage_submitted = app.submitted_date is not None
        app.stage_under_review = app.status in ['Submitted', 'Under_Review']
        app.stage_docs_verified = app.status in ['Approved', 'Rejected']
        app.stage_ward_approved = app.status == 'Approved'
        app.stage_amount_awarded = app.approved_amount is not None

        # Get approvals
        app.approvals_list = list(app.approvals.all().order_by('-approved_date'))

    context = {
        'applications': applications,
    }

    return render(request, 'applications/track_application.html', context)
```

---

4. FRONTEND IMPLEMENTATION

4.1 Base Template

File: `templates/base.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Bursary Management System{% endblock %}</title>

    <!-- Bootstrap CSS -->
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
    <!-- Font Awesome -->
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css">
    <!-- Custom CSS -->
    <link rel="stylesheet" href="{% static 'css/style.css' %}">

    {% block extra_css %}{% endblock %}
</head>
<body>
    <!-- Navigation Bar -->
    <nav class="navbar navbar-expand-lg navbar-dark bg-primary">
        <div class="container">
            <a class="navbar-brand" href="{% url 'home' %}">
```

```
                <i class="fas fa-graduation-cap"></i> Bursary System
            </a>

            <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#navbarNav">
                <span class="navbar-toggler-icon"></span>
            </button>

            <div class="collapse navbar-collapse" id="navbarNav">
                <ul class="navbar-nav ms-auto">
                    {% if user.is_authenticated %}
                        {% if user.admin_role %}
                            <!-- Admin Menu -->
                            <li class="nav-item">
                                <a class="nav-link" href="{% url
'admin_panel:dashboard' %}">
                                    Dashboard
                                </a>
                            </li>
                            <li class="nav-item">
                                <a class="nav-link" href="{% url
'admin_panel:applications_review' %}">
                                    Applications
                                </a>
                            </li>
                        {% else %}
                            <!-- User Menu -->
                            <li class="nav-item">
                                <a class="nav-link" href="{% url 'users:dashboard'
%}">
                                    Dashboard
                                </a>
                            </li>
                            <li class="nav-item">
                                <a class="nav-link" href="{% url
'applications:new_application' %}">
                                    New Application
                                </a>
                            </li>
                        {% endif %}

                        <li class="nav-item dropdown">
                            <a class="nav-link dropdown-toggle" href="#"
role="button" data-bs-toggle="dropdown">
                                <i class="fas fa-user"></i> {{ user.get_full_name
}}
                            </a>
                            <ul class="dropdown-menu">
                                <li><a class="dropdown-item" href="{% url
'users:profile' %}">Profile</a></li>
                                <li><hr class="dropdown-divider"></li>
                                <li><a class="dropdown-item" href="{% url
'users:logout' %}">Logout</a></li>
                            </ul>
```

```
                        </li>
                    {% else %}
                        <li class="nav-item">
                            <a class="nav-link" href="{% url 'users:login'
%}">Login</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link" href="{% url 'users:register'
%}">Register</a>
                        </li>
                    {% endif %}
                </ul>
            </div>
        </div>
    </nav>

    <!-- Alert Messages -->
    <div class="container mt-3">
        {% if messages %}
            {% for message in messages %}
                <div class="alert alert-{{ message.tags }} alert-dismissible fade
show" role="alert">
                    {{ message }}
                    <button type="button" class="btn-close" data-bs-
dismiss="alert"></button>
                </div>
            {% endfor %}
        {% endif %}
    </div>

    <!-- Main Content -->
    <main class="py-4">
        {% block content %}{% endblock %}
    </main>

    <!-- Footer -->
    <footer class="bg-dark text-white text-center py-3 mt-5">
        <div class="container">
            <p>&copy; 2026 Matungu Subcounty Bursary Management System. All rights
reserved.</p>
        </div>
    </footer>

    <!-- Bootstrap JS -->
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"
></script>
    <!-- jQuery -->
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
    <!-- Custom JS -->
    <script src="{% static 'js/main.js' %}"></script>

    {% block extra_js %}{% endblock %}
```

```
</body>
</html>
```

4.2 Login Template

File: `templates/users/login.html`

```
{% extends 'base.html' %}
{% load static %}

{% block title %}Login - Bursary System{% endblock %}

{% block content %}
<div class="container">
    <div class="row justify-content-center">
        <div class="col-lg-5 col-md-8">
            <div class="card shadow-lg">
                <div class="card-body p-5">
                    <h2 class="card-title text-center mb-4">
                        <i class="fas fa-sign-in-alt"></i> Login
                    </h2>

                    <!-- Login Form -->
                    <form method="POST" action="{% url 'users:login' %}">
                        {% csrf_token %}

                        <!-- Username -->
                        <div class="mb-3">
                            <label for="username" class="form-label">Username or
Email:</label>
                            <input type="text"
                                   class="form-control {% if form.username.errors
%}is-invalid{% endif %}"
                                   id="username"
                                   name="username"
                                   required>
                            {% if form.username.errors %}
                                <div class="invalid-feedback">{{
form.username.errors }}</div>
                            {% endif %}
                        </div>

                        <!-- Password -->
                        <div class="mb-3">
                            <label for="password" class="form-label">Password:
</label>
                            <input type="password"
                                   class="form-control {% if form.password.errors
%}is-invalid{% endif %}"
                                   id="password"
                                   name="password"
```

```
                                    required>
                                {% if form.password.errors %}
                                    <div class="invalid-feedback">{{
form.password.errors }}</div>
                                {% endif %}
                            </div>

                            <!-- Remember Me (optional) -->
                            <div class="mb-3 form-check">
                                <input type="checkbox" class="form-check-input"
id="remember" name="remember">
                                <label class="form-check-label"
for="remember">Remember me</label>
                            </div>

                            <!-- Login Button -->
                            <button type="submit" class="btn btn-primary w-100 mb-3">
                                <i class="fas fa-sign-in-alt"></i> Login
                            </button>
                        </form>

                        <!-- Divider -->
                        <hr>

                        <!-- Links -->
                        <p class="text-center mb-0">
                            Don't have an account?
                            <a href="{% url 'users:register' %}">Register here</a>
                        </p>

                        <p class="text-center">
                            <a href="{% url 'users:forgot_password' %}">Forgot
password?</a>
                        </p>
                    </div>
                </div>
            </div>
        </div>
</div>
{% endblock %}
```

4.3 Application Form Template

File: `templates/applications/new_application_step1.html`

```
{% extends 'base.html' %}
{% load static %}

{% block title %}New Application - Step 1 - Bursary System{% endblock %}

{% block content %}
```

```
<div class="container">
    <div class="row justify-content-center">
        <div class="col-lg-8">
            <!-- Progress Bar -->
            <div class="mb-4">
                <div class="progress" style="height: 30px;">
                    <div class="progress-bar" role="progressbar" style="width:
25%">
                        Step 1 of 4: Personal Information
                    </div>
                </div>
            </div>

            <!-- Form Card -->
            <div class="card shadow">
                <div class="card-header bg-primary text-white">
                    <h4 class="mb-0">Personal Information</h4>
                </div>

                <div class="card-body">
                    <form method="POST" action="{% url
'applications:new_application' %}?step=1">
                        {% csrf_token %}

                        <!-- Date of Birth -->
                        <div class="mb-3">
                            <label for="date_of_birth" class="form-label">Date of
Birth *</label>
                            <input type="date"
                                   class="form-control"
                                   id="date_of_birth"
                                   name="date_of_birth"
                                   required>
                            <small class="text-muted">Must be 18 years or
older</small>
                        </div>

                        <!-- Gender -->
                        <div class="mb-3">
                            <label for="gender" class="form-label">Gender *
</label>
                            <select class="form-control" id="gender" name="gender"
required>
                                <option value="">Select Gender</option>
                                <option value="Male">Male</option>
                                <option value="Female">Female</option>
                            </select>
                        </div>

                        <!-- National ID -->
                        <div class="mb-3">
                            <label for="national_id" class="form-label">National
ID *</label>
                            <input type="text"
```

```
                                    class="form-control"
                                    id="national_id"
                                    name="national_id"
                                    placeholder="12345678"
                                    required>
                                <small class="text-muted">For fraud prevention - must
be unique</small>
                            </div>

                            <!-- Marital Status -->
                            <div class="mb-3">
                                <label for="marital_status" class="form-label">Marital
Status *</label>
                                <select class="form-control" id="marital_status"
name="marital_status" required>
                                    <option value="Single">Single</option>
                                    <option value="Married">Married</option>
                                </select>
                            </div>

                            <!-- Orphan Status -->
                            <div class="mb-3 form-check">
                                <input type="checkbox"
                                       class="form-check-input"
                                       id="is_orphan"
                                       name="is_orphan">
                                <label class="form-check-label" for="is_orphan">
                                    I am an orphan (both parents deceased)
                                </label>
                            </div>

                            <!-- Navigation Buttons -->
                            <div class="d-flex justify-content-between mt-4">
                                <button type="button" class="btn btn-secondary"
onclick="window.history.back()">
                                    <i class="fas fa-arrow-left"></i> Back
                                </button>
                                <button type="submit" class="btn btn-primary">
                                    Next <i class="fas fa-arrow-right"></i>
                                </button>
                            </div>
                        </form>
                    </div>
                </div>
            </div>
        </div>
</div>
{% endblock %}
```

4.4 Dashboard Template

File: `templates/users/dashboard.html`

```
{% extends 'base.html' %}
{% load static %}

{% block title %}Dashboard - Bursary System{% endblock %}

{% block content %}
<div class="container">
    <!-- Welcome Section -->
    <div class="row mb-4">
        <div class="col-lg-12">
            <div class="card bg-primary text-white">
                <div class="card-body">
                    <h2>Welcome, {{ user.get_full_name }}!</h2>
                    <p class="mb-0">Manage your bursary applications here</p>
                </div>
            </div>
        </div>
    </div>

    <!-- Statistics -->
    <div class="row mb-4">
        <div class="col-md-3">
            <div class="card text-center">
                <div class="card-body">
                    <h3>{{ stats.total }}</h3>
                    <p class="text-muted">Total Applications</p>
                </div>
            </div>
        </div>
        <div class="col-md-3">
            <div class="card text-center">
                <div class="card-body">
                    <h3>{{ stats.approved }}</h3>
                    <p class="text-success">Approved</p>
                </div>
            </div>
        </div>
        <div class="col-md-3">
            <div class="card text-center">
                <div class="card-body">
                    <h3>{{ stats.under_review }}</h3>
                    <p class="text-warning">Under Review</p>
                </div>
            </div>
        </div>
        <div class="col-md-3">
            <div class="card text-center">
                <div class="card-body">
                    <h3>{{ stats.rejected }}</h3>
                    <p class="text-danger">Rejected</p>
                </div>
            </div>
```

```
        </div>
    </div>

    <!-- Quick Actions -->
    <div class="row mb-4">
        <div class="col-lg-12">
            <a href="{% url 'applications:new_application' %}" class="btn btn-
success btn-lg">
                <i class="fas fa-plus-circle"></i> New Application
            </a>
            <a href="{% url 'applications:track_application' %}" class="btn btn-
info btn-lg">
                <i class="fas fa-search"></i> Track Applications
            </a>
        </div>
    </div>

    <!-- Applications List -->
    <div class="row">
        <div class="col-lg-12">
            <div class="card">
                <div class="card-header">
                    <h5 class="mb-0">Your Applications</h5>
                </div>
                <div class="table-responsive">
                    <table class="table table-hover mb-0">
                        <thead class="table-light">
                            <tr>
                                <th>School</th>
                                <th>Program</th>
                                <th>Status</th>
                                <th>Submitted</th>
                                <th>Amount Awarded</th>
                                <th>Actions</th>
                            </tr>
                        </thead>
                        <tbody>
                            {% for app in applications %}
                                <tr>
                                    <td>{{ app.school.name }}</td>
                                    <td>{{ app.program.name }}</td>
                                    <td>
                                        {% if app.status == 'Draft' %}
                                            <span class="badge bg-
secondary">Draft</span>
                                        {% elif app.status == 'Submitted' %}
                                            <span class="badge bg-
info">Submitted</span>
                                        {% elif app.status == 'Under_Review' %}
                                            <span class="badge bg-warning">Under
Review</span>
                                        {% elif app.status == 'Approved' %}
                                            <span class="badge bg-
success">Approved</span>
```

```
                                        {% elif app.status == 'Rejected' %}
                                            <span class="badge bg-
danger">Rejected</span>
                                        {% endif %}
                                    </td>
                                    <td>
                                        {% if app.submitted_date %}
                                            {{ app.submitted_date|date:"M d, Y" }}
                                        {% else %}
                                            -
                                        {% endif %}
                                    </td>
                                    <td>
                                        {% if app.approved_amount %}
                                            KES {{
app.approved_amount|floatformat:2 }}
                                        {% else %}
                                            -
                                        {% endif %}
                                    </td>
                                    <td>
                                        <a href="{% url
'applications:view_application' app.id %}"
                                           class="btn btn-sm btn-primary">
                                            View
                                        </a>
                                    </td>
                                </tr>
                            {% empty %}
                                <tr>
                                    <td colspan="6" class="text-center text-muted
py-4">
                                        No applications yet. <a href="{% url
'applications:new_application' %}">Create one</a>
                                    </td>
                                </tr>
                            {% endfor %}
                        </tbody>
                    </table>
                </div>
            </div>
        </div>
    </div>
</div>
{% endblock %}
```

---

5. AUTHENTICATION & AUTHORIZATION

5.1 Permission Decorators

File: `users/decorators.py`

```python
from functools import wraps
from django.shortcuts import redirect
from django.contrib import messages
import logging

logger = logging.getLogger(__name__)

def login_required_custom(view_func):
    """Custom login required decorator"""
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not request.user.is_authenticated:
            messages.warning(request, 'Please login first.')
            return redirect('users:login')
        return view_func(request, *args, **kwargs)
    return wrapper


def admin_required(role_type=None):
    """Check if user is admin with optional role check"""
    def decorator(view_func):
        @wraps(view_func)
        def wrapper(request, *args, **kwargs):
            if not request.user.is_authenticated:
                messages.error(request, 'Please login first.')
                return redirect('users:login')

            if not hasattr(request.user, 'admin_role'):
                messages.error(request, 'Admin access required.')
                return redirect('users:dashboard')

            admin_role = request.user.admin_role

            if not admin_role.is_active:
                messages.error(request, 'Your admin account is inactive.')
                return redirect('users:login')

            if role_type and admin_role.role_type != role_type:
                messages.error(request, f'{role_type} access required.')
                return redirect('admin_panel:dashboard')

            return view_func(request, *args, **kwargs)
        return wrapper
    return decorator


def applicant_required(view_func):
    """Check if user is applicant (not admin)"""
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not request.user.is_authenticated:
            messages.warning(request, 'Please login first.')
```

```
            return redirect('users:login')

        if hasattr(request.user, 'admin_role'):
            messages.error(request, 'Only applicants can access this page.')
            return redirect('admin_panel:dashboard')

        return view_func(request, *args, **kwargs)
    return wrapper
```

5.2 Form Validation

File: `users/forms.py`

```
from django import forms
from django.contrib.auth.models import User
from .models import UserProfile, AdminRole
from applications.models import Ward
import re

class RegisterForm(forms.Form):
    """User registration form with validation"""

    username = forms.CharField(
        max_length=150,
        required=True,
        widget=forms.TextInput(attrs={
            'class': 'form-control',
            'placeholder': 'Username'
        })
    )
    email = forms.EmailField(
        required=True,
        widget=forms.EmailInput(attrs={
            'class': 'form-control',
            'placeholder': 'Email address'
        })
    )
    first_name = forms.CharField(
        max_length=30,
        required=False,
        widget=forms.TextInput(attrs={
            'class': 'form-control',
            'placeholder': 'First name'
        })
    )
    last_name = forms.CharField(
        max_length=30,
        required=False,
        widget=forms.TextInput(attrs={
            'class': 'form-control',
            'placeholder': 'Last name'
```

```
        })
    )
    national_id = forms.CharField(
        max_length=20,
        required=True,
        widget=forms.TextInput(attrs={
            'class': 'form-control',
            'placeholder': 'National ID (e.g., 12345678)'
        })
    )
    phone_number = forms.CharField(
        max_length=20,
        required=False,
        widget=forms.TextInput(attrs={
            'class': 'form-control',
            'placeholder': '+254...'
        })
    )
    password = forms.CharField(
        min_length=8,
        required=True,
        widget=forms.PasswordInput(attrs={
            'class': 'form-control',
            'placeholder': 'Password (min 8 characters)'
        })
    )
    password_confirm = forms.CharField(
        min_length=8,
        required=True,
        widget=forms.PasswordInput(attrs={
            'class': 'form-control',
            'placeholder': 'Confirm password'
        })
    )
    gender = forms.ChoiceField(
        required=False,
        choices=[('', 'Select Gender'), ('Male', 'Male'), ('Female', 'Female')],
        widget=forms.Select(attrs={'class': 'form-control'})
    )

    def clean_username(self):
        """Validate username is unique"""
        username = self.cleaned_data['username']
        if User.objects.filter(username=username).exists():
            raise forms.ValidationError('Username already exists.')
        return username

    def clean_email(self):
        """Validate email is unique"""
        email = self.cleaned_data['email']
        if User.objects.filter(email=email).exists():
            raise forms.ValidationError('Email already registered.')
        return email
```

```
    def clean_national_id(self):
        """Validate national ID format and uniqueness"""
        national_id = self.cleaned_data['national_id'].strip()

        # Check format (must be numeric)
        if not national_id.isdigit():
            raise forms.ValidationError('National ID must contain only numbers.')

        # Check if already registered (FRAUD PREVENTION)
        if UserProfile.objects.filter(national_id=national_id).exists():
            raise forms.ValidationError('This National ID is already registered.')

        return national_id

    def clean(self):
        """Validate password confirmation"""
        cleaned_data = super().clean()
        password = cleaned_data.get('password')
        password_confirm = cleaned_data.get('password_confirm')

        if password != password_confirm:
            raise forms.ValidationError('Passwords do not match.')

        return cleaned_data


class AdminRoleAssignmentForm(forms.Form):
    """Assign admin role to user"""

    ROLE_CHOICES = (
        ('Ward_Admin', 'Ward Admin'),
        ('CDF_Admin', 'CDF Admin'),
        ('Super_Admin', 'Super Admin'),
    )

    user = forms.ModelChoiceField(
        queryset=User.objects.filter(admin_role__isnull=True),
        widget=forms.Select(attrs={'class': 'form-control'})
    )
    role_type = forms.ChoiceField(
        choices=ROLE_CHOICES,
        widget=forms.Select(attrs={'class': 'form-control'})
    )
    ward = forms.ModelChoiceField(
        queryset=Ward.objects.filter(is_active=True),
        required=False,
        widget=forms.Select(attrs={'class': 'form-control'})
    )

    def clean(self):
        """Validate ward is selected for Ward Admin"""
        cleaned_data = super().clean()
        role_type = cleaned_data.get('role_type')
        ward = cleaned_data.get('ward')
```

```
        if role_type == 'Ward_Admin' and not ward:
            raise forms.ValidationError('Ward Admin must have a ward assigned.')

        return cleaned_data
```

---

6. FILE STRUCTURE & ORGANIZATION

6.1 Complete Directory Structure

```
Bursary_system/
│
├── bursary_system/                # Main Django Project
│   ├── __init__.py
│   ├── settings.py                ← Django configuration
│   ├── urls.py                    ← URL routing
│   ├── asgi.py
│   └── wsgi.py
│
├── users/                         # User Management App
│   ├── migrations/
│   │   ├── 0001_initial.py
│   │   ├── 0002_add_verification.py
│   │   └── ...
│   ├── models.py                  ← User models
│   ├── views.py                   ← Auth views
│   ├── forms.py                   ← User forms
│   ├── urls.py                    ← User URLs
│   ├── decorators.py              ← Permission decorators
│   ├── admin.py
│   ├── apps.py
│   └── tests.py
│
├── applications/                  # Application Management
│   ├── migrations/
│   ├── models.py                  ← Application models
│   ├── views.py                   ← Application views
│   ├── forms.py                   ← Application forms
│   ├── urls.py
│   ├── admin.py
│   ├── apps.py
│   └── tests.py
│
├── schools/                       # School Management
│   ├── migrations/
│   ├── models.py                  ← School models
│   ├── views.py
│   ├── urls.py
│   ├── admin.py
│   └── apps.py
```

```
    │
    ├── admin_panel/                    # Admin Dashboard
    │   ├── migrations/
    │   ├── models.py
    │   ├── views.py                    ← Admin views
    │   ├── urls.py
    │   ├── admin.py
    │   └── apps.py
    │
    ├── notifications/                  # Notification System
    │   ├── models.py
    │   ├── views.py
    │   ├── tasks.py                    ← Celery tasks
    │   ├── urls.py
    │   └── admin.py
    │
    ├── templates/                      # HTML Templates
    │   ├── base.html                   ← Base template
    │   ├── users/
    │   │   ├── login.html
    │   │   ├── register.html
    │   │   ├── dashboard.html
    │   │   ├── profile.html
    │   │   └── ...
    │   ├── applications/
    │   │   ├── new_application_step1.html
    │   │   ├── new_application_step2.html
    │   │   ├── new_application_step3.html
    │   │   ├── new_application_step4.html
    │   │   ├── upload_documents.html
    │   │   ├── track_application.html
    │   │   └── ...
    │   ├── admin_panel/
    │   │   ├── dashboard.html
    │   │   ├── applications_for_review.html
    │   │   ├── approve_application.html
    │   │   ├── reject_application.html
    │   │   ├── manage_admins.html
    │   │   ├── assign_admin_role.html
    │   │   └── ...
    │   └── schools/
    │       └── schools_list.html
    │
    ├── static/                         # Static Files
    │   ├── css/
    │   │   ├── style.css               ← Custom CSS
    │   │   └── bootstrap.min.css
    │   ├── js/
    │   │   ├── main.js                 ← Custom JS
    │   │   └── forms.js
    │   └── images/
    │       └── logo.png
    │
    ├── media/                          # User Uploads
```

```
│   └── applications/
│       └── documents/
│
├── logs/                          # Application Logs
│   └── debug.log
│
├── manage.py                      ← Django management
├── requirements.txt               ← Dependencies
├── .env                          ← Environment variables
├── .env.example                  ← Env template
├── .gitignore                    ← Git ignore rules
└── db.sqlite3                    ← SQLite database (dev)
```

---

7. COMPLETE CODE EXAMPLES

7.1 URL Configuration

File: `bursary_system/urls.py`

```python
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static
from django.views.generic import TemplateView

urlpatterns = [
    # Admin
    path('admin/', admin.site.urls),

    # Apps
    path('users/', include('users.urls')),
    path('applications/', include('applications.urls')),
    path('schools/', include('schools.urls')),
    path('admin-panel/', include('admin_panel.urls')),
    path('notifications/', include('notifications.urls')),

    # Home
    path('', TemplateView.as_view(template_name='home.html'), name='home'),
]

# Serve media files in development
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
    urlpatterns += static(settings.STATIC_URL, document_root=settings.STATIC_ROOT)
```

File: `users/urls.py`

```
from django.urls import path
from . import views

app_name = 'users'

urlpatterns = [
    # Authentication
    path('register/', views.register, name='register'),
    path('login/', views.login_view, name='login'),
    path('logout/', views.logout_view, name='logout'),
    path('verify-email/<str:token>/', views.verify_email, name='verify_email'),

    # Password Reset
    path('forgot-password/', views.forgot_password, name='forgot_password'),
    path('reset-password/', views.reset_password, name='reset_password'),

    # User Dashboard & Profile
    path('dashboard/', views.dashboard, name='dashboard'),
    path('profile/', views.profile, name='profile'),

    # Admin Management
    path('assign-admin/', views.assign_admin_role, name='assign_admin_role'),
    path('manage-admins/', views.manage_admins, name='manage_admins'),
]
```

File: `applications/urls.py`

```
from django.urls import path
from . import views

app_name = 'applications'

urlpatterns = [
    # Application Management
    path('new/', views.new_application, name='new_application'),
    path('<str:id>/view/', views.view_application, name='view_application'),
    path('<str:id>/edit/', views.edit_application, name='edit_application'),
    path('<str:id>/submit/', views.submit_application, name='submit_application'),

    # Documents
    path('<str:id>/documents/', views.upload_documents, name='upload_documents'),

    # Tracking
    path('track/', views.track_application, name='track_application'),
]
```

7.2 Custom CSS Styling

File: `static/css/style.css`

```
/* Variables */
:root {
    --primary-color: #0d6efd;
    --secondary-color: #6c757d;
    --success-color: #198754;
    --danger-color: #dc3545;
    --warning-color: #ffc107;
    --info-color: #0dcaf0;
    --light-bg: #f8f9fa;
    --border-radius: 0.5rem;
}

/* Global Styles */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background-color: #f5f5f5;
    color: #333;
}

/* Cards */
.card {
    border: none;
    box-shadow: 0 0.125rem 0.25rem rgba(0, 0, 0, 0.075);
    border-radius: var(--border-radius);
}

.card:hover {
    box-shadow: 0 0.5rem 1rem rgba(0, 0, 0, 0.15);
    transition: box-shadow 0.3s ease;
}

/* Buttons */
.btn {
    border-radius: var(--border-radius);
    font-weight: 500;
    transition: all 0.3s ease;
}

.btn-primary {
    background-color: var(--primary-color);
    border-color: var(--primary-color);
}

.btn-primary:hover {
    background-color: #0b5ed7;
    border-color: #0b5ed7;
```

```
}

/* Forms */
.form-control {
    border-radius: var(--border-radius);
    border: 1px solid #dee2e6;
    padding: 0.6rem 1rem;
}

.form-control:focus {
    border-color: var(--primary-color);
    box-shadow: 0 0 0 0.2rem rgba(13, 110, 253, 0.25);
}

.form-label {
    font-weight: 500;
    color: #555;
    margin-bottom: 0.5rem;
}

/* Tables */
.table {
    border-collapse: collapse;
}

.table thead {
    background-color: var(--light-bg);
}

.table tbody tr:hover {
    background-color: var(--light-bg);
}

/* Badges */
.badge {
    padding: 0.5rem 0.75rem;
    border-radius: var(--border-radius);
    font-weight: 500;
}

/* Alerts */
.alert {
    border-radius: var(--border-radius);
    border: none;
}

/* Navigation */
.navbar {
    box-shadow: 0 0.125rem 0.25rem rgba(0, 0, 0, 0.075);
}

/* Status Indicators */
.status-draft {
    color: var(--secondary-color);
```

```
}

.status-submitted {
    color: var(--info-color);
}

.status-approved {
    color: var(--success-color);
}

.status-rejected {
    color: var(--danger-color);
}

/* Responsive */
@media (max-width: 768px) {
    .table-responsive {
        font-size: 0.875rem;
    }

    .btn {
        padding: 0.4rem 0.8rem;
        font-size: 0.875rem;
    }
}
```

---

8. TESTING & VALIDATION

8.1 Unit Tests

File: `users/tests.py`

```
from django.test import TestCase, Client
from django.contrib.auth.models import User
from .models import UserProfile, AdminRole
from applications.models import Ward
import logging

logger = logging.getLogger(__name__)

class UserRegistrationTestCase(TestCase):
    """Test user registration functionality"""

    def setUp(self):
        """Set up test client"""
        self.client = Client()

    def test_register_new_user(self):
        """Test successful user registration"""
        response = self.client.post('/users/register/', {
```

```
            'username': 'testuser',
            'email': 'test@example.com',
            'first_name': 'Test',
            'last_name': 'User',
            'national_id': '12345678',
            'phone_number': '+254123456789',
            'gender': 'Male',
            'password': 'testpassword123',
            'password_confirm': 'testpassword123',
        })

        # Check user created
        self.assertTrue(User.objects.filter(username='testuser').exists())

        # Check profile created with national ID
        user = User.objects.get(username='testuser')
        self.assertTrue(hasattr(user, 'profile'))
        self.assertEqual(user.profile.national_id, '12345678')

    def test_duplicate_national_id_prevention(self):
        """Test fraud prevention - duplicate national ID"""
        # Create first user
        user1 = User.objects.create_user(
            username='user1',
            email='user1@example.com',
            password='pass123'
        )
        UserProfile.objects.create(
            user=user1,
            national_id='12345678'
        )

        # Try to create second user with same ID
        response = self.client.post('/users/register/', {
            'username': 'user2',
            'email': 'user2@example.com',
            'national_id': '12345678',  # Duplicate
            'password': 'password123',
            'password_confirm': 'password123',
        })

        # Should fail
        self.assertEqual(User.objects.filter(username='user2').count(), 0)


class AdminRoleTestCase(TestCase):
    """Test admin role assignment"""

    def setUp(self):
        """Create test users"""
        self.super_admin = User.objects.create_user(
            username='superadmin',
            email='super@example.com',
            password='pass123'
```

```
        )
        AdminRole.objects.create(
            user=self.super_admin,
            role_type='Super_Admin'
        )

        self.ward_obj = Ward.objects.create(
            name='Test Ward',
            county='Test County'
        )

    def test_admin_role_assignment(self):
        """Test admin role creation"""
        user = User.objects.create_user(
            username='wardadmin',
            email='admin@example.com',
            password='pass123'
        )

        admin_role = AdminRole.objects.create(
            user=user,
            role_type='Ward_Admin',
            ward=self.ward_obj
        )

        self.assertEqual(admin_role.role_type, 'Ward_Admin')
        self.assertEqual(admin_role.ward, self.ward_obj)

    def test_admin_permissions(self):
        """Test admin permissions"""
        user = User.objects.create_user(
            username='admin',
            email='admin@example.com',
            password='pass123'
        )

        admin_role = AdminRole.objects.create(
            user=user,
            role_type='CDF_Admin'
        )

        permissions = admin_role.get_permissions()
        self.assertIn('view_all_applications', permissions)
        self.assertIn('approve_applications', permissions)
```

8.2 Integration Tests

```
class ApplicationSubmissionTestCase(TestCase):
    """Test complete application workflow"""

    def setUp(self):
```

```
        """Create test user and school"""
        self.client = Client()
        self.user = User.objects.create_user(
            username='applicant',
            email='applicant@example.com',
            password='pass123'
        )
        UserProfile.objects.create(
            user=self.user,
            national_id='87654321'
        )

        from schools.models import School, Program
        self.school = School.objects.create(
            name='Test University',
            school_type='University'
        )
        self.program = Program.objects.create(
            school=self.school,
            name='Computer Science',
            category='Engineering'
        )

    def test_complete_application_workflow(self):
        """Test full application submission"""
        # Login
        self.client.login(username='applicant', password='pass123')

        # Create application (step by step)
        # This would test the full multi-step form
```

SUMMARY

This guide covers the complete development of the Bursary Management System:

- ☑ **Database Design** - Normalized schema with relationships
- ☑ **Backend** - Django models, views, and authentication
- ☑ **Frontend** - Bootstrap templates and forms
- ☑ **Security** - Authentication, authorization, fraud prevention
- ☑ **Testing** - Unit and integration tests

Each section includes complete, production-ready code that can be implemented step-by-step.

For Questions or Issues:

- Check the development logs in `logs/debug.log`
- Review Django documentation at https://docs.djangoproject.com/
- Test each component before moving to the next phase
- Use `python manage.py test` to validate functionality